

SME Operating Agent Technical Documentation

Architecture Reasoning Memory Safety and Evaluation

Company	Stellar Ark AI
Team code	<missing>
Hackathon	Show Me Your Agents
Repository	github.com/joshualumzy/personal_agent_long_context
Document status	Evidence-grounded draft
Snapshot commit	<missing>
Date	27 September 2026

Document purpose

Explain how the implemented prototype retrieves company evidence, carries user-scoped memory, converts meeting transcripts into controlled follow-up, and supports founder-led recruiting while keeping consequential actions under human control.

Technical Summary

The SME Operating Agent is a TypeScript and Fastify modular monolith with three browser surfaces: Company Context, Meeting Actions, and Recruiting. The workflows share identity, model access, company evidence, and human-control patterns, but keep Company Evidence, Personal Memory, meeting state, and recruiting state in distinct stores and contracts.

The primary runtime pattern is controlled model reasoning around typed tools. Models may select retrieval or workflow tools; deterministic server code validates inputs, screens prohibited content, checks citations, assigns action risk tiers, binds approval to an exact payload, and controls which effects can leave the application. The system does not give a model arbitrary database access or an unrestricted workplace-write tool.

Personas and Operating Context

Persona	Technical role in the prototype	Boundary
Stellar Ark AI employee	Asks company questions and reviews meeting follow-up	Target user; private company records are not included in the hackathon data
Founder or hiring manager	Confirms role criteria, reviews candidates and completes outreach	Human judgment remains authoritative for hiring and sending
OrgForge employee persona	Provides a privacy-safe session identity and employee-visible demo corpus	Synthetic demonstration and evaluation data only
Administrator	Configures credentials, providers, database and optional integrations	Production SSO and fine-grained authorization are not claimed

The current roster contains 51 synthetic OrgForge employees. Jax, Priya, Chloe, Marcus, and Deepa are pinned as convenient demonstration identities. A signed session is the source of employee identity; a request cannot silently substitute a different employee identifier.

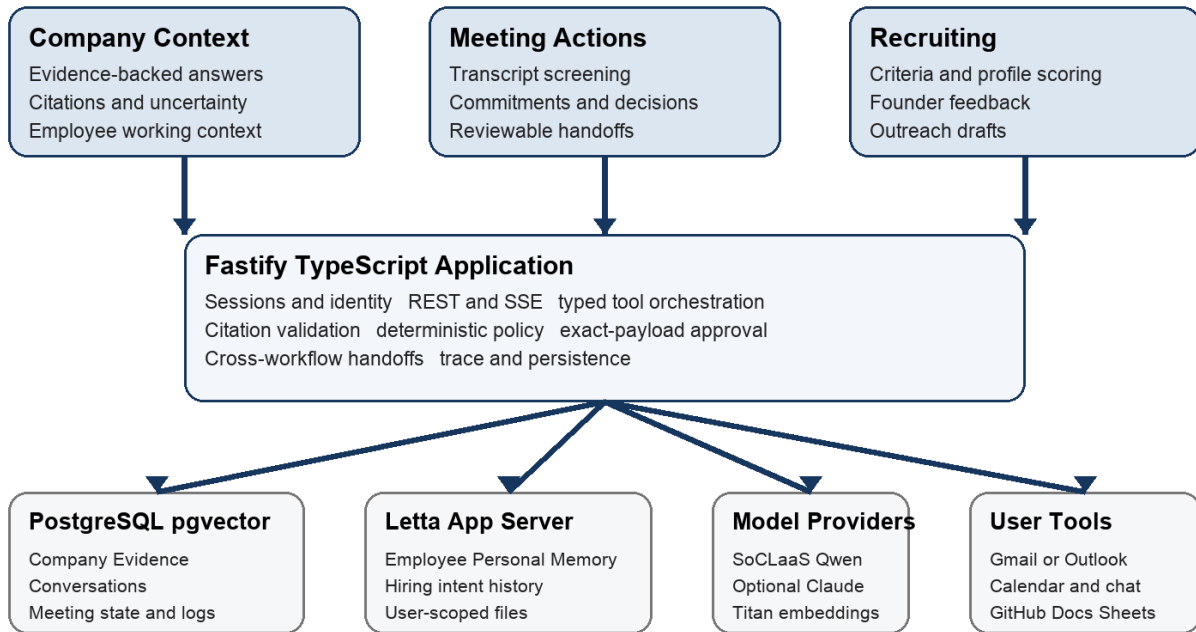
Implemented Product Surfaces

Surface	Route	Purpose
Company Context	/	Evidence-backed company questions, source inspection, conversation history and Personal Memory
Meeting Actions	/meetings	Transcript intake, action extraction, decision conflicts, approval and handoff
Recruiting	/recruiting	Role criteria, candidate pool, founder feedback, outreach and follow-up

Architecture Design

SME Operating Agent Runtime Architecture

One modular monolith with three browser workflows and separated evidence, memory, and action state



The application is intentionally a modular monolith for the hackathon. A single Fastify process owns HTTP validation, sessions, orchestration, model calls, tool dispatch, and Server-Sent Events. Domain modules isolate the three workflows without adding distributed-system overhead that the prototype does not need.

Component Responsibilities

Layer	Responsibility
Browser	Plain HTML, CSS and JavaScript experiences for chat, meeting work and recruiting; the main chat can embed a recruiting panel
Fastify application	Authentication, validation, REST and SSE, conversation context, model selection, errors and persistence
Domain services	S1 reasoning loop, S2 meeting state machine and policy, and S3 recruiting state machine
Stores	PostgreSQL and pgvector for evidence and operational state; Letta for user-scoped Personal Memory
Providers	SoCLaaS Qwen by default, an optional Claude-compatible gateway, Bedrock Titan embeddings, and optional workflow APIs

Frameworks Models and Tools

Category	Technology	Use in this system
Application	Node.js 22.19 or newer TypeScript Fastify	Modular server, validated endpoints, sessions and static browser delivery
Reasoning models	NUS SoCLaaS Qwen qwen3.8:27b by default	Company retrieval, meeting extraction and drafting, and recruiting judgments
Optional model	Claude through the configured OpenAI-compatible gateway	Selectable Company Context model when gateway credentials are present
Embeddings	Amazon Titan Text Embeddings V2	1,024-dimensional Company Evidence vectors stored in pgvector
Evidence store	PostgreSQL full-text search and pgvector	Keyword and semantic retrieval, evidence chunks, links, conversations and meeting state
Long-term memory	Self-hosted Letta App Server	Employee-scoped working context and founder hiring-intent history
Optional recruiting services	Exa Hunter Prospeo Gmail and LinkedIn inbox reader	Public profile search, work-email lookup, read-only reply input and manual user handoff
Optional meeting helpers	Google free-busy and Jev	Availability checks and a fast reader for times or decision conflicts with model fallback
Quality tooling	Node test runner JSDOM Playwright TypeScript and offline evaluators	Domain, browser, holdout, regression and live-provider checks

Model and Agent Design

- S1 is a bounded tool-using loop. It normally allows up to four reasoning steps, or eight when workflow skills are registered, and forces a final answer on the last step.
- The first S1 step requires a tool call. Available company tools are typed `search_company_knowledge` and `get_related_sources`; recruiting tools remain unavailable until the recruiting skill is loaded.
- S2 uses a model to extract candidates and draft payloads, but deterministic code validates transcript quotes, risk tiers, approvals and executable effects.
- S3 represents hiring as explicit role, criterion, candidate, proposal and message state rather than relying on free-form chat history alone.
- No multi-agent orchestration is claimed. The architecture is one application with specialised domain services and a lazily loaded recruiting skill.

Data State and Memory Design

Short-Term Working State

State	Implementation	Purpose
Current request	Validated request body and authenticated employee session	Defines the active user, question and selected model
Conversation window	Latest six stored user or assistant turns, each clipped to 4,000 characters	Supports follow-up questions without loading an unbounded transcript
S1 run state	In-memory retrieved-source map, tool-call list and run identifier	Ensures citations can name only sources retrieved in that run
S2 processing state	Queued transcript batches and persisted MeetingState	Prevents per-line model lag and tracks actions, decisions and trace events
S3 role state	One JSON document per role behind a RoleRepository interface	Keeps criteria, candidates, feedback, proposals, messages and simulated time together

Long-Term Memory

Letta stores Personal Memory, not the Company Evidence corpus. Each employee identifier maps to a separate Letta agent tag. Memory file access is confined to the resolved Memory directory and a restricted tool allow-list. Inspection excludes persona, skill and Git metadata files. Background memory updates are serialized per employee to avoid race conditions.

Personal Memory keeps concise user context with source and recorded time. Corrections can supersede earlier entries, cancellations remain visible as history, and unresolved contradictions are retained as conflicts. This context may help interpret a question, but it is displayed separately and cannot serve as a Company Evidence citation.

Storage Boundaries

Store	Contains	Must not contain
PostgreSQL	Company Artifacts, chunks, embeddings, explicit links, conversations, employees, meetings and action logs	Evaluation Oracle records excluded by ingestion policy
Letta	Employee Personal Memory and recruiting intent events	OrgForge Company Evidence corpus
Recruiting role files	Per-role operational state and public professional profile fields	Guessed email addresses or unrelated private profile data
Offline evaluation	Expected answers, labels and reserved Oracle data	Runtime tool access

Company Context Reasoning Loop

1. Authenticate the employee and reject any explicit user identifier that conflicts with the signed session.
2. Load the latest bounded conversation window and the employee's separately labelled Personal Memory when available.
3. Require a typed tool call to search Company Evidence. The model may run focused keyword or semantic searches and may traverse explicit links only from sources already retrieved.
4. Accumulate retrieved evidence in a server-side map and synthesise a concise response from that evidence and the session profile.
5. Validate every citation identifier against the map. If repair cannot produce a supported answer, return an insufficient-evidence outcome instead of fabricating a source.
6. Persist the turn with model, provider, duration, run identifier, sources, tool calls, memory status and outcome; stream progress and final validated content over SSE when requested.

Retrieval Design

Mechanism	Implementation	Control
Keyword retrieval	PostgreSQL tsvector full-text search over chunks	Employee-visible admitted artifact types only
Semantic retrieval	Cosine similarity over Titan V2 vectors in pgvector	Embedding model recorded with each vector
Graph traversal	document_links between explicit OrgForge artifact references	Seeds must already be present in the current retrieved map
Citation delivery	Inline source identifiers and inspectable source metadata	Unknown identifiers are removed or repaired before delivery

The runtime corpus currently contains 4,966 synthetic employee-visible OrgForge artifacts across Slack, email, Confluence, Jira, Zoom transcripts, pull requests, Salesforce and Zendesk. OrgForge is privacy-safe demonstration and evaluation data, not Stellar Ark AI business data or a product dependency.

Meeting Actions Reasoning and Human Control

1. Accept transcript text typed, pasted, replayed from a scripted scenario, or loaded from an admitted Zoom transcript. Speech-to-text is outside the prototype.
2. Screen each segment before model use. Prohibited data is withheld from storage; documented prompt-injection shapes become blocked actions and never reach extraction.
3. Extract commitments, company questions and decisions from new segments only. A candidate is discarded if its quoted trigger does not occur in the transcript.
4. Answer read-only company questions through S1, compare decisions with earlier meeting decisions, and retrieve evidence or missing contact and calendar fields for drafts.
5. Apply deterministic policy, deduplicate repeated commitments, persist the action and append trace and status events.
6. For approval-tier work, bind approval to the payload hash. Any edit increments the version, changes the hash and returns the action to proposed state.
7. Execute only a user-controlled handoff, an explicitly simulated record, or a draft recruiting role. Escalated and blocked actions are rejected again by the executor as defence in depth.

Risk-Calibrated Autonomy

Tier	Examples	Runtime behavior
Automatic	Evidence-backed answer and decision conflict	Read-only result is produced immediately and traced
Approval	Email message calendar ticket document spreadsheet or hiring draft	Employee approves the exact unchanged payload before handoff
Escalate	Money price refund discount or contract commitment	Named higher authority required; employee cannot execute
Blocked	Secret or instruction attack	No model processing and no executor path

Everyday effects open as prefilled drafts in the employee's own Gmail or Outlook, calendar, WhatsApp or Teams, GitHub, document, or spreadsheet surface. The external tool's own signed-in user completes the send or save, so the prototype does not hold broad workplace-write credentials.

Recruiting Reasoning and Human Control

1. Turn a typed need, uploaded job description, or known profile link into three to six must or nice-to-have criteria; the founder reviews and confirms them before search.
2. Use Exa public-profile search when configured or clearly labelled fictional sample profiles otherwise. Judge each criterion as yes, no or unclear with a short profile-grounded reason.
3. Assign a deterministic candidate tier from the criterion verdicts and present the pool in the Recruiting workspace or an embedded main-chat panel.
4. Treat founder feedback as explicit state. Repeated pass reasons may create a criterion proposal, and a stalled search may create a widening proposal; neither applies until the founder accepts it.
5. For a chosen candidate, try Hunter and then Prospeo for a work email. The system does not guess an address. Draft outreach in the founder's voice and warn if it repeats a private pass reason.
6. Read replies through configured Gmail, a read-only LinkedIn inbox sync, or pasted text; draft scheduling or follow-up and retain the founder's explicit hiring rationale in scoped memory.
7. Erase closed candidate records after the configured 30-day retention period in the current prototype state model.

Recruiting Control Points

Decision	Agent contribution	Required human action
Role criteria	Proposes and explains criteria	Founder edits and confirms
Candidate assessment	Scores declared criteria from public profile fields	Founder reviews keeps or passes
Preference learning	Proposes a criterion from repeated feedback	Founder accepts or declines
Search expansion	Proposes the next widening step after inactivity	Founder accepts or declines
Outreach	Finds a non-guessed work email and drafts text	Founder resolves warnings and sends from their own account
Hiring outcome	Tracks status and rationale	Founder makes the decision

Guardrails Security and Constraints

Control area	Implemented control	Current claim boundary
Identity	HMAC-SHA256 session tokens, HttpOnly SameSite cookies and script password verification	Prototype identity; enterprise SSO and fine-grained department authorization are deferred
Input safety	Request-size validation, prohibited-data detection and S2 pre-model injection screening	No claim of complete prompt-injection resistance
Evidence integrity	Runtime source admission rules, Oracle exclusion and current-run citation validation	A valid citation identifier does not alone prove perfect semantic support
Memory isolation	Per-employee Letta agent tags, confined memory tools and separate display	Production correction deletion and tenant-governance controls are deferred
Action safety	Deterministic S2 tiers, payload-hash approval, edit invalidation and executor defence in depth	No unattended consequential action or shared multi-party approval
Recruiting privacy	Public professional fields only, no guessed email, private-reason warnings and retention cleanup	Legal basis fairness assessment and production privacy review remain deployment responsibilities
Browser security	Content Security Policy, no-referrer, nosniff and frame denial headers	External penetration testing is <missing>
Secrets	Server-side environment configuration; credentials are not sent to the browser or tracked	Deployment secret manager and rotation evidence are <missing>

Prototype Constraints

- The hackathon demo does not use private Stellar Ark AI records. Quantitative business impact still requires a controlled internal pilot.
- Meeting input is text. Audio capture and speech-to-text are outside the implemented surface.
- External writes are user-controlled handoffs or clearly marked simulations; existing third-party records are not edited through provider APIs.
- Optional services change the available path. Missing Exa uses labelled sample candidates; missing email lookup leaves outreach without an address; missing Google access keeps reply and availability features manual.
- Deployment URL, production hosting evidence, final provider configuration and submission snapshot commit are <missing>.

Observability and Evaluation

Operational Observability

- S1 persists model, provider, duration, run identifier, retrieved sources, tool calls, memory status, outcome and conversation persistence status with each assistant turn.
- S2 emits live segment, action, trace and busy events and persists the full meeting state. meeting_action_log is an append-only record of action kind, tier, status, payload hash, time and structured detail.
- Application failure records carry operation, correlation identifier, user identifier and reason; transcript, question and Memory content are intentionally excluded from that failure shape.
- Evaluation Oracle data and expected answers remain in offline runners that the runtime agent cannot call.

Saved Evaluation Evidence

Area	Saved evidence	Interpretation
Company Context	76-question run using qwen3.8:27b: 100 percent citation integrity, 29 percent expected-evidence recall, 12 percent factual accuracy, 39.68 seconds mean latency	Supports the citation-identifier guard in that sample; answer quality and retrieval recall remain improvement targets
Meeting Actions	One run dated 25 September 2026: 23 of 23 tiers correct, 9 of 9 unsafe lines blocked, 0 of 44 ordinary lines blocked, 0 actions in 29 negative cases, 1 of 1 conflict found	Strong bounded guardrail evidence; action-kind recall was complete except email at 2 of 3
Personal Memory	Pilot over 20 selected mutation-heavy weekly Memora questions across two personas	Development gate only; not a full benchmark or production reliability claim
Recruiting	Deterministic, holdout, regression, chaos and browser test assets exist	Final externally reportable submission snapshot result is <missing>

Submission Verification Commands

Command	Purpose
npm run typecheck	TypeScript contract check
npm test	Unit and regression suite
npm run test:browser	Browser rendering and interaction check
npm run test:orgforge	Runtime corpus and Oracle-isolation check
npm run test:holdout and npm run test:lock	Recruiting holdout and regression-lock checks
npm run eval:orgforge meeting or recruiting	Live model evaluation paths

A fresh clean verification run against the final submission snapshot is pending. The current checkout's typecheck cannot resolve several declared packages because the local dependency installation is incomplete; this draft therefore does not claim a new all-green run.

Deployment and Operations

Current Runtime Topology

Runtime	Required configuration	Purpose
Fastify process	Node.js 22.19 or newer, session secret, database URL and model credentials	Serves all three browser surfaces and APIs
PostgreSQL with pgvector	Docker Compose or equivalent managed database	Evidence, embeddings, conversations, employee roster, meetings and logs
Letta App Server	Local or protected service endpoint and optional token	Personal Memory and hiring-intent persistence
Model endpoint	SoCLaaS key or configured compatible gateway	Reasoning and structured model calls
AWS Bedrock	Region ap-southeast-2 and authenticated profile	Titan Text Embeddings V2 backfill and optional model path
Optional workflow services	Service-specific keys or Google OAuth	Recruiting search, contact lookup, replies and calendar availability

Local Start Sequence

1. Install declared Node and Python dependencies and create environment-specific secrets.
2. Start PostgreSQL, apply migrations, ingest the declared synthetic corpus, and backfill embeddings when semantic retrieval is required.
3. Start the Letta App Server and the Fastify application.
4. Open the root company chat, the meetings route, or the recruiting route and confirm health, identity and configured provider status.
5. Run deterministic verification and the selected live evaluation before recording deployment evidence or the demo video.

Deployment Evidence Status

Evidence item	Status
Public or judge-accessible deployment URL	<missing>
Hosting architecture and region	<missing>
Health-check capture	<missing>
Database migration and corpus counts	Local commands defined; final deployed evidence <missing>
Configured model and optional integrations used in the demo	<missing>
Submission snapshot commit	<missing>

Implementation Traceability

Concern	Primary implementation	Verification or evidence
HTTP sessions SSE and conversations	src/http-app.ts src/auth.ts src/adapters/postgres-conversations.ts	test/auth.test.ts test/conversations.test.ts test/sme-browser.test.ts
S1 reasoning and citations	src/soclaas-company-agent.ts src/adapters/postgres-company-knowledge.ts	test/soclaas-company-agent.test.ts test/unified-agent.test.ts eval/orgforge
Personal Memory	src/adapters/letta-memory.ts src/memory-queue.ts	test/letta-memory.test.ts test/memory-updates.test.ts eval/memora
Meeting extraction and policy	src/meetings/extractor.ts guard.ts policy.ts service.ts	test/meetings-core.test.ts test/meetings-routes.test.ts eval/meetings
Approval and handoff	src/meetings/service.ts executor.ts handoff.ts	test/meetings-store-executor.test.ts test/meetings-handoff.test.ts
Recruiting state and tools	src/recruiting/service.ts routes.ts chat-tools.ts roles.ts	test/recruiting.test.ts test/holdout test/regression eval/recruiting
Schema and storage	database/migrations scripts/orgforge	npm run db:migrate npm run test:orgforge
Browser surfaces	public/index.html app.js meetings.html meetings.js recruiting.html recruiting.js	test/sme-browser.test.ts and browser-specific regression assets

Technical Conclusion

The prototype demonstrates a coherent context-to-action architecture: evidence retrieval is inspectable, Personal Memory is separately scoped, meeting work is risk-tiered and approval-bound, and recruiting remains founder-controlled. Its strongest design choice is that model reasoning operates inside deterministic technical boundaries rather than replacing them. The remaining submission work is operational evidence: restore the declared dependency installation, run the final verification suite, record the deployed configuration, and replace the remaining <missing> fields with the exact submission snapshot.